Algorithms and Data Structures

Gabi Röger, Patrick Schnider
Spring Semester 2026

University of Basel
Computer Science

Exercise sheet 3

Due: Friday, March 20, 2026

Please upload a single PDF of your solution to ADAM. If you wish to upload a new version of your handin, please make sure that it has the same file name and is uploaded from the same account.

Exercise 3.1 (asymptotic notation, 0.5 + 0.5 + 0.5 + 0.5 marks)
Which of the following statements are correct? Justify your answer (no formal proof needed).

(a) If $f \in O(n^2)$, then $f \in \Omega(n)$.

(b) $\Theta(n^k) \cap \Theta(n^{k'}) = \emptyset$ for $k < k'$

(c) if $f \in \Theta(g)$ and $g \in \Theta(h)$ then $h \in \Theta(f)$

(d) if $f \in O(g)$ and $f \in O(h)$ then $g \in O(h)$.

Exercise 3.2 (Θ, 1 + 1 + 1 marks)
We consider the complexity classes $\Theta(1), \Theta(\log_2 n), \Theta(n), \Theta(n \log_2 n), \Theta(n^2)$ and $\Theta(n^3)$. In which of these classes are the following algorithms? Justify your answer (no formal proof needed).
Hint: $x\%y$ computes the remainder of dividing x by y. For example, $7\%3 = 1$.

(a)
```
1  def kame(n, array):
2      for i in range(n): # i = 0, ..., n-1
3          sum = 0
4          for j in range(n-i): # i = 0, ..., n-i-1
5              sum = sum + array[i + j]
6          array[i] = sum
```

(b)
```
1  def hame(n):
2      while n%10 != 0:
3          n = n - 1
4      return n/10
```

(c)
```
1  def ha(n):
2      while n > 0:
3          print(n%2)
4          n = n//2
5      return res
```

Exercise 3.3 (runtime analysis for recursive algorithms, 1 + 3 + 1 marks)
The following algorithm searches an entry `value` in a sorted array between positions `low` and `high`. It returns the position of one such entry if it exists and `None` otherwise.
All parts of this exercise consider the *worst case* running time in terms of $n = \texttt{high} - \texttt{low} + 1$. In the algorithm, the worst case is that `array` does not contain `value`, so that the algorithm never returns in line 10.

```python
def binary_search(array, value, low, high):
    if high == low and array[low] == value:
        return low
    if high <= low:
        return None

    # compute position roughly in the middle between high and low
    mid = low + (high - low)//2
    if array[mid] == value:
        return mid
    if array[mid] > value:
        return binary_search(array, value, low, mid)
        # we would usually exclude position mid in this recursive
        # call (because we know that array[mid] != value) but
        # include it in this exercise to keep it simpler.
    else:
        return binary_search(array, value, mid+1, high)
```

(a) Specify a master recurrence of the form $T(n) = A \cdot T(n/B) + f(n)$ for `binary_search`. This means, you need to identify suitable numbers $A \geq 1$ and $B > 1$ as well as a suitable function f (or alternatively the asymptotic growth of f in Θ notation).

As in the lecture, you may assume that the input size is a power of 2, so you don't need to consider any floor or ceiling operations. This won't have an impact on the asymptotic growth.

(b) Consider classes $O(1), O(\log_2 n), O(n), O(n \log n)$. Use the substitution method (Ch. A11) to guess the tightest of these classes (the one representing the slowest asymptotic growth) that contains the solution of $T(n)$ from part (a), and prove by mathematical induction that your proof is correct (you may omit the base case).

(c) Use the master theorem (Ch. A11) to identify the growth of T from part (a) in Θ notation. Which of the three cases is applicable? What can you conclude for the growth of T?